PROYECTO DE SISTEMAS DIGITALES

Joystick analógico para controlar un videojuego 2D

Equipo: **Teknical May Cry**

Integrantes

Federico Mattia · Máximo Orue · Ulises Ferrari · Lucas del Pino

Informe técnico: componentes, lógica de funcionamiento, programación y articulación con Unity

Documento de proyecto

1. Introducción

Este proyecto consiste en construir un sistema de control basado en un joystick analógico y dos botones conectados a un Arduino UNO. El objetivo es utilizar esos controles para manejar un personaje dentro de un videojuego 2D desarrollado en Unity. El joystick permite indicar una dirección y los botones agregan acciones, como interactuar con objetos y abrir o cerrar el inventario.

La propuesta integra electrónica y programación: el circuito recibe las acciones físicas del usuario, Arduino interpreta las señales y las convierte en mensajes, y Unity lee esos mensajes para actualizar el comportamiento del juego.

2. Objetivos

• Construir y probar un circuito con joystick analógico y dos pulsadores.
• Leer las señales del joystick y detectar las pulsaciones con Arduino.
• Evitar el movimiento diagonal, priorizando una sola dirección por vez.
• Comunicar Arduino con Unity mediante el puerto serie USB.
• Usar los botones para interactuar y alternar el inventario.
• Relacionar los contenidos de sistemas digitales, sensores, entradas, condiciones y eventos de programación.

3. Componentes utilizados

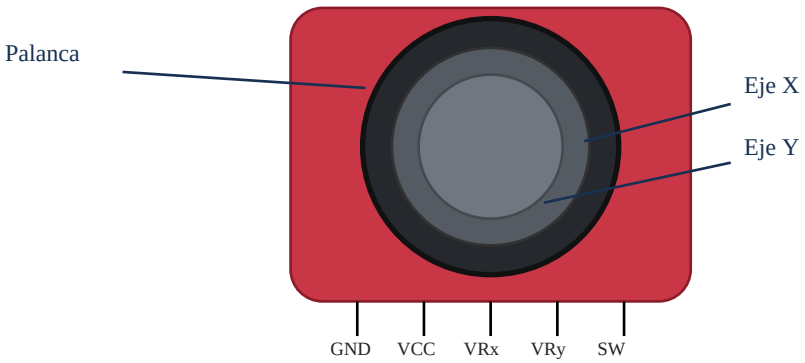
Arduino UNO: placa que lee las entradas y ejecuta el programa de control.
Joystick analógico: módulo con dos ejes (X e Y) y, normalmente, un pulsador integrado. Sus ejes entregan valores analógicos según la posición de la palanca.
Dos pulsadores: se utilizan como botón de interacción y botón de inventario.
Cables de conexión: unen los componentes con los pines de alimentación y señal.
Computadora: ejecuta Unity y se conecta al Arduino por USB.
Wokwi: entorno de simulación donde se puede verificar el circuito y el programa antes de probarlo con el hardware real.

4. ¿Cómo funciona un joystick analógico?

El joystick posee dos ejes que permiten medir la inclinación horizontal y vertical. En Arduino UNO, **analogRead()** devuelve valores entre 0 y 1023. Cuando la palanca está centrada, cada eje suele dar un valor cercano a 512; al moverla, los valores aumentan o disminuyen.

El programa define umbrales para distinguir una dirección. En este proyecto se toma como referencia: Y mayor que 600 = arriba; Y menor que 400 = abajo; X menor que 400 = izquierda; X mayor que 600 = derecha. Si ninguno supera esos límites, el personaje queda quieto. Los umbrales crean una zona central para evitar movimientos involuntarios por pequeñas variaciones de lectura.

Imagen de referencia del joystick analógico



Esquema ilustrativo de un módulo típico. La distribución exacta de pines puede variar según el modelo; conviene verificar las etiquetas impresas en el módulo real.

5. Conexiones del circuito

Componente / pin	Conexión en Arduino UNO	Función
Joystick GND	GND	Referencia eléctrica
Joystick VCC	5V	Alimentación
Joystick HORZ / VRx	A0	Lectura horizontal
Joystick VERT / VRy	A1	Lectura vertical
Pulsador 1	D2 y GND	Interactuar
Pulsador 2	D3 y GND	Inventario

Los botones se configuran con **INPUT_PULLUP**. En ese modo, el pin normalmente lee HIGH y al presionar el botón conectado a GND lee LOW. Por eso el programa detecta una pulsación cuando el valor cambia de HIGH a LOW. Este método evita tener que añadir una resistencia externa para cada botón, si la conexión está hecha correctamente.

6. Lógica del sistema

Lectura: Arduino consulta los dos ejes analógicos y el estado de los dos botones.
Clasificación: compara X e Y con los umbrales de 400 y 600.
Prioridad direccional: usa una cadena de condiciones if / else if. Así solo se comunica una dirección por ciclo, sin diagonales. En el código, la prioridad es arriba, abajo, izquierda y derecha.
Detección de flanco: se guarda el estado anterior de cada botón. La acción se activa solamente al detectar el paso de HIGH a LOW, en lugar de repetirse continuamente mientras el botón permanece apretado.
Inventario: una variable booleana alterna entre abierto y cerrado cada vez que se presiona el segundo botón.
Comunicación: Arduino envía líneas de texto por el puerto serie a 9600 baudios; Unity interpreta esos mensajes.

7. Cómo se articula con el proyecto de programación

Unity es el entorno donde se ejecuta el videojuego. El Arduino funciona como un dispositivo de entrada externo. La conexión USB permite que el Arduino aparezca como un puerto serie (por ejemplo, COM3 en Windows; el número real depende de la computadora). Un script C# abre ese puerto, lee los mensajes y convierte cada orden en una acción del juego.

El protocolo propuesto utiliza mensajes claros: **MOVE:UP**, **MOVE:DOWN**, **MOVE:LEFT**, **MOVE:RIGHT** y **MOVE:STOP** para el movimiento; **ACTION:INTERACT** para interactuar; y **ACTION:INVENTORY** para alternar el

inventario. Es importante que el código de Arduino y el de Unity utilicen exactamente los mismos mensajes.

La lógica de interacción y del inventario del juego debe conectarse con los sistemas que ya existan en Unity. En el ejemplo incluido, las acciones se muestran en la consola con Debug.Log; eso sirve para comprobar la comunicación, pero después se reemplaza por las funciones reales del juego.

8. Código de Arduino

Este código implementa el protocolo compatible con el script de Unity que aparece en la siguiente sección.

```
const int ejeX = A0;
const int ejeY = A1;
const int botonInteractuar = 2;
const int botonInventario = 3;

bool anteriorInteractuar = HIGH;
bool anteriorInventario = HIGH;
bool inventarioAbierto = false;
String direccionAnterior = "";

void setup() {
  Serial.begin(9600);
  pinMode(botonInteractuar, INPUT_PULLUP);
  pinMode(botonInventario, INPUT_PULLUP);
}

void loop() {
  int x = analogRead(ejeX);
  int y = analogRead(ejeY);
  String direccion = "STOP";

  // Solo se envía una dirección a la vez: no hay diagonales.
  if (y > 600) direccion = "UP";
  else if (y < 400) direccion = "DOWN";
  else if (x < 400) direccion = "LEFT";
  else if (x > 600) direccion = "RIGHT";

  // Enviar movimiento solo cuando cambia, para reducir mensajes.
  if (direccion != direccionAnterior) {
    Serial.print("MOVE:");
    Serial.println(direccion);
    direccionAnterior = direccion;
  }

  bool actualInteractuar = digitalRead(botonInteractuar);
  bool actualInventario = digitalRead(botonInventario);

  // INPUT_PULLUP: HIGH = suelto, LOW = presionado.
  if (anteriorInteractuar == HIGH && actualInteractuar == LOW) {
    Serial.println("ACTION:INTERACT");
  }

  if (anteriorInventario == HIGH && actualInventario == LOW) {
    inventarioAbierto = !inventarioAbierto;
    Serial.println("ACTION:INVENTORY");
  }

  anteriorInteractuar = actualInteractuar;
  anteriorInventario = actualInventario;
  delay(30);
}
```

9. Código de Unity (C#)

Crear un script llamado **ArduinoController.cs** y agregarlo al objeto del personaje. El ejemplo mueve al personaje en el plano XY. Cambiar COM3 por el puerto real del Arduino. Si el juego usa otro sistema de movimiento, habrá que adaptar este ejemplo al controlador existente.

```
using UnityEngine;
using System;
using System.IO.Ports;

public class ArduinoController : MonoBehaviour
{
    public float velocidad = 5f;

    private SerialPort arduino;
    private string direccion = "STOP";

    void Start()
    {
        arduino = new SerialPort("COM3", 9600);
        arduino.ReadTimeout = 50;

        try
        {
            arduino.Open();
        }
        catch (Exception e)
        {
            Debug.LogError("No se pudo conectar al Arduino: " + e.Message);
        }
    }

    void Update()
    {
        if (arduino != null && arduino.IsOpen)
        {
            try
            {
                // Leer una línea por vez; Arduino termina cada mensaje con salto de línea.
                string mensaje = arduino.ReadLine().Trim();

                if (mensaje.StartsWith("MOVE:"))
                    direccion = mensaje.Substring(5);

                if (mensaje == "ACTION:INTERACT")
                    Debug.Log("Interactuar: conectar aquí la lógica del juego.");

                if (mensaje == "ACTION:INVENTORY")
                    Debug.Log("Inventario: alternar aquí la interfaz del juego.");
            }
            catch (TimeoutException) { }
            catch (Exception e)
            {
                Debug.LogWarning("Error leyendo el puerto serie: " + e.Message);
            }
        }

        Vector3 movimiento = Vector3.zero;

        switch (direccion)
        {
            case "UP":    movimiento = Vector3.up; break;
            case "DOWN":  movimiento = Vector3.down; break;
            case "LEFT":  movimiento = Vector3.left; break;
            case "RIGHT": movimiento = Vector3.right; break;
        }

        transform.position += movimiento * velocidad * Time.deltaTime;
    }

    void OnApplicationQuit()
    {
        if (arduino != null && arduino.IsOpen)
            arduino.Close();
    }
}
```

Importante: el ejemplo usa movimiento directo de Transform para que se entienda la comunicación. Si el personaje usa Rigidbody2D, colisiones o un controlador ya programado, el movimiento debe integrarse con ese sistema en lugar de mover el Transform directamente. No abrir al mismo tiempo el Monitor Serie del IDE de Arduino y Unity, porque normalmente solo una aplicación puede usar el puerto.

10. Imagen del circuito armado en Wokwi

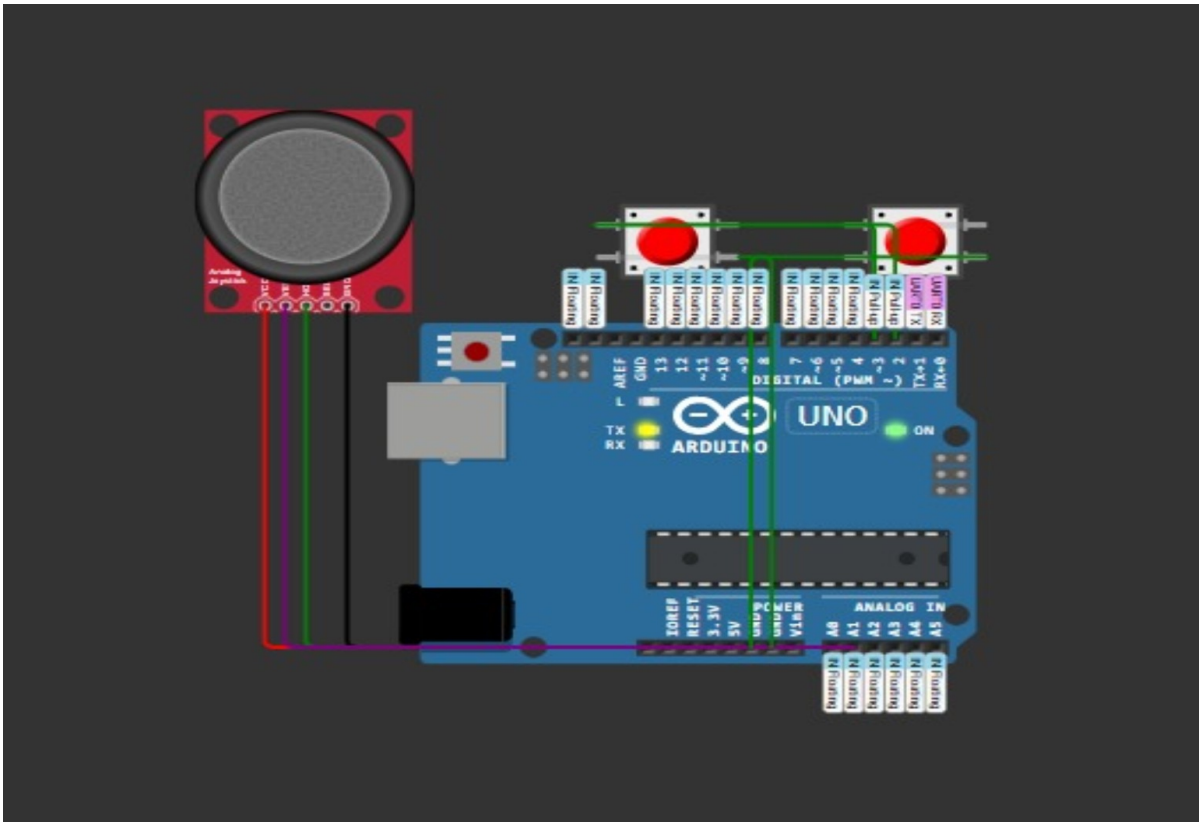


Imagen aportada por el equipo: simulación del Arduino UNO, joystick analógico y dos pulsadores. Revisar que las conexiones coincidan con la tabla de este informe antes de replicar el circuito físicamente.

11. Pruebas y verificación

1. En Wokwi, comprobar que al mover el joystick se reconoce una dirección por vez.
2. Confirmar que al dejar el joystick en el centro se envía MOVE:STOP.
3. Presionar el botón de interacción y verificar ACTION:INTERACT una sola vez por pulsación.
4. Presionar el botón de inventario y verificar ACTION:INVENTORY.
5. Con el Arduino físico, cargar el programa y conectarlo por USB. Identificar el puerto serie correcto.
6. En Unity, configurar ese puerto y comprobar primero los mensajes en la consola. Luego conectar las acciones a los sistemas reales del juego.

12. Limitaciones y aspectos a tener en cuenta

La simulación de Wokwi permite probar el circuito y el código, pero no necesariamente crea un puerto COM local que Unity pueda leer directamente. Para una conexión serial directa, lo habitual es usar el Arduino físico conectado por USB. Además, la lectura de puerto serie depende del sistema operativo y de la configuración de Unity; si **System.IO.Ports** no está disponible en la configuración del proyecto, habrá que ajustar los parámetros de compatibilidad o utilizar una alternativa serial compatible.

Los umbrales de 400 y 600 son valores iniciales y pueden necesitar ajustes según el módulo. También conviene probar las direcciones porque algunos joysticks tienen el eje Y invertido respecto de lo esperado; si ocurre, se intercambian las condiciones de arriba y abajo.

13. Conclusión

El proyecto Teknical May Cry propone una forma práctica de conectar un dispositivo electrónico con un videojuego. El joystick transforma el movimiento de la mano en valores analógicos; Arduino interpreta esos valores y las pulsaciones de los botones; y Unity recibe mensajes que se traducen en movimiento e interacción. El trabajo reúne conceptos de electrónica, entradas digitales y analógicas, lógica condicional, variables de estado, comunicación serie y programación orientada a objetos. Como siguiente etapa, se deben integrar las acciones de interacción e inventario con los sistemas definitivos del videojuego y validar el funcionamiento con el Arduino físico.

14. Resumen de responsabilidades de cada parte

Joystick: indica dirección mediante dos señales analógicas.
Pulsadores: generan órdenes de interacción e inventario.
Arduino: lee, clasifica y envía mensajes por USB.
Unity: interpreta mensajes y actualiza el juego.
Equipo: diseña, conecta, programa, prueba y documenta el sistema.